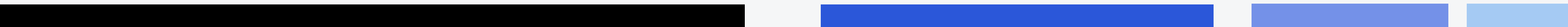


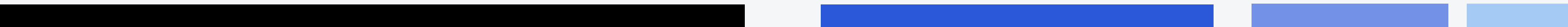
Lecture-2

Python - Introduction



Environment Variables

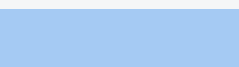
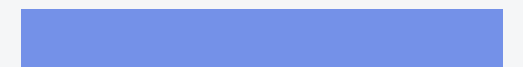
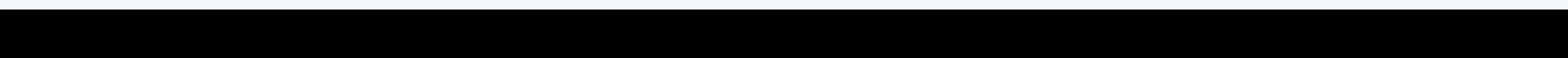
- Environment variables in Windows act as system-wide settings that store key system information for the OS and your apps.
- Instead of hardcoding details like file paths or temporary storage locations directly into software, applications check these labelled values at runtime.
- For example, PATH tells Windows where to look for executable files so you can run tools like Python from any terminal.



Environment Variables

Windows splits these into two tiers:

- **System variables**, which apply to every user account on the machine and require admin privileges to alter.
- **User variables**, which only affect your specific account. If the two ever conflict, your user-level settings take priority.



Python Module

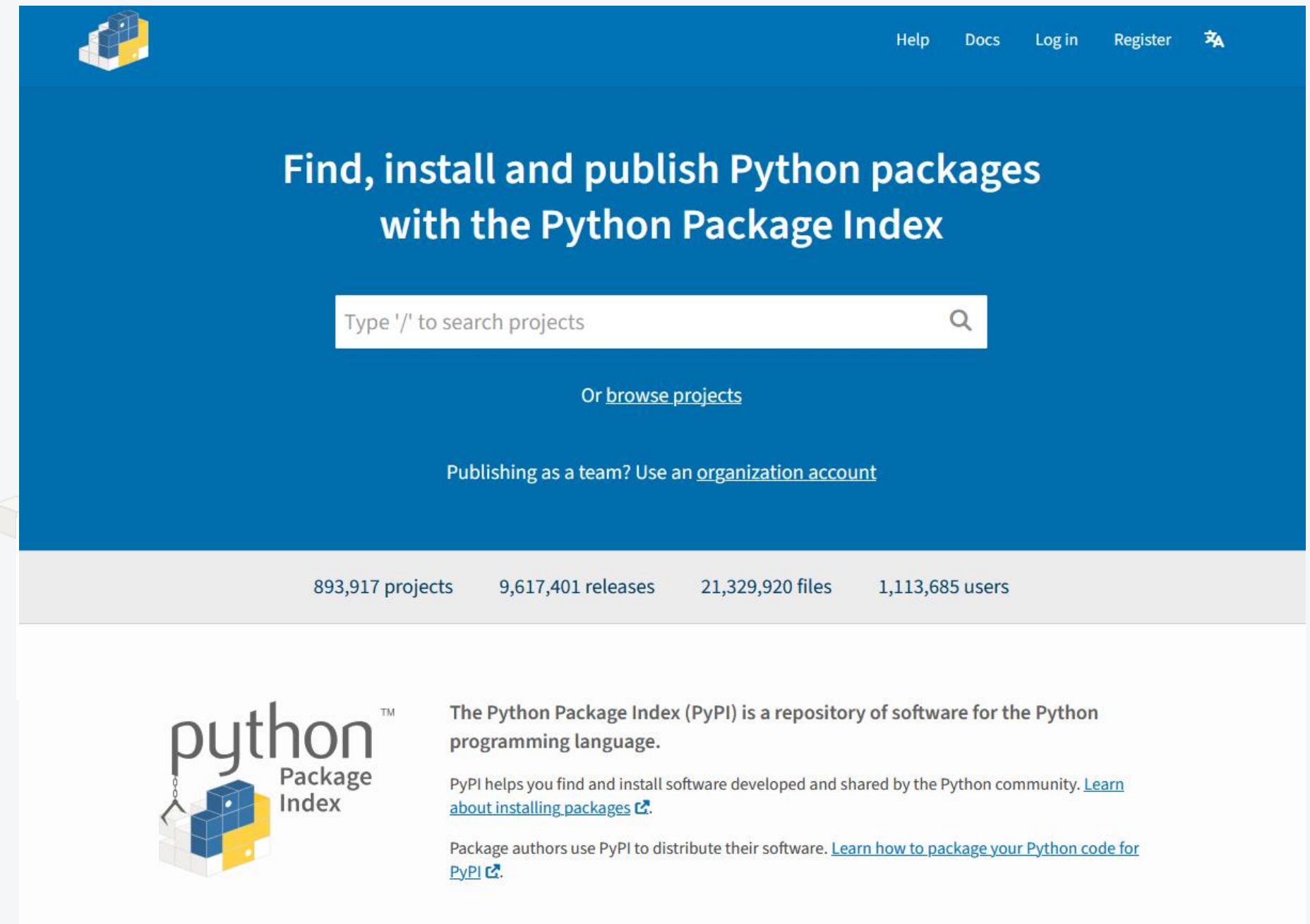
A simple Python file that contains collections of classes and functions.

Python Package

- A directory containing related modules and sub-packages that acts as a single importable unit.
- Package requires an `__init__.py` file (which can be empty, execute setup code, or define public exports).
- It represents a folder structure so importing `mypackage` allows you to access nested components like `mypackage.submodule`.

Python Package Index (PyPI)

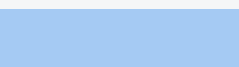
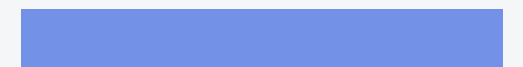
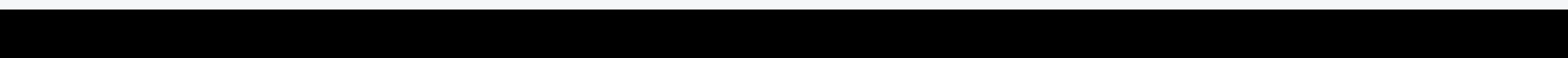
- The official public repository for the Python language.
- Developers from around the world publish and share reusable Python code packages.
- PyPI serves two main audiences:
Package Consumers:
Package Publishers:



Python Package Index (PyPI)

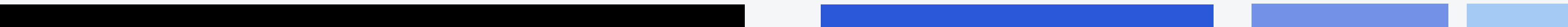
Key Elements Hosted on PyPI

- **Metadata:** Descriptive information about the packages, including version numbers, author details, licenses, and compatibility requirements.
- **Wheels (.whl):** Pre-compiled package distributions that install almost instantly.
- **Source Distributions (.tar.gz):** Raw source code files used if a pre-compiled version isn't available for your specific system.



pip

- The most popular tool for installing Python packages, and the one included with modern versions of Python.
- It provides the essential core features for finding, downloading, and installing packages from PyPI and other Python package indexes, and can be incorporated into a wide range of development workflows via its command-line interface (CLI).



Virtual Environments

“Virtual Environments” allow Python [packages](#) to be installed in an isolated location for a particular application, rather than being installed globally.

- Imagine you have an application that needs version 1 of LibFoo, but another application requires version 2. How can you use both these applications?
- If you install everything into `/usr/lib/python3.6/site-packages`, it's easy to end up in a situation where you unintentionally upgrade an application that shouldn't be upgraded.

Requirements for Installing Packages

1. Ensure you can run Python from the command line

Unix/macOS Windows

```
py --version
```

2. Ensure you can run pip from the command line

Unix/macOS Windows

```
py -m pip --version
```

3. Ensure pip is up to date

Unix/macOS Windows

```
py -m pip install --upgrade pip
```

Requirements for Installing Packages

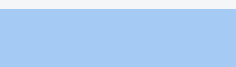
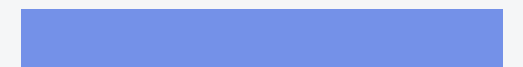
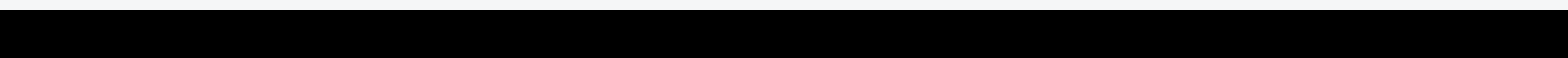
4. Optionally, create and/or activate a virtual environment

Unix/macOS **Windows**

```
py -m venv tutorial_env  
tutorial_env\Scripts\activate
```

5. Install the package. **pip install numpy**
6. Deactivating the virtual env. **deactivate**

Python DataTypes



List

Set

Tuple

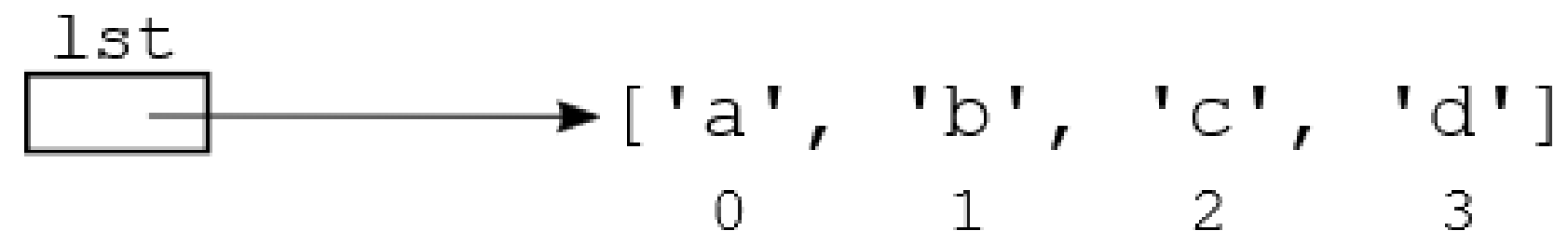
Dictionary

List

- A list contains series of any data type: strings, ints, other lists.
- The things inside a list are generically called "elements".
- Lists are "mutable"

```
>>> lst = ['a', 'b', 'c', 'd']
```

Here is what that list looks like in memory

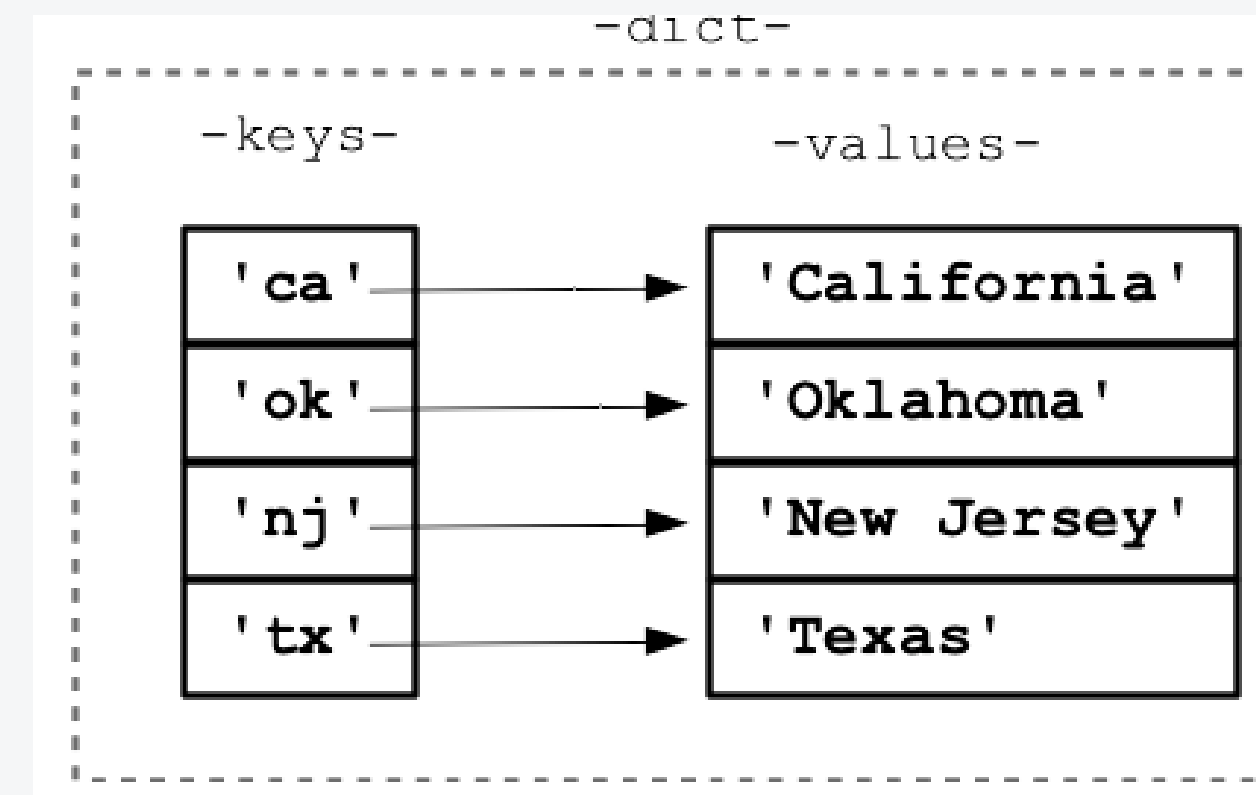


List Basic Operations

- `lst = []` - **create an empty list**
- `lst = [1, 2, 3]` - **create a list with data in it**
- `lst.append(value)` - add *value* to the end of a list, increasing the list's length by 1.
- `len(lst)` - **access length of a list**
- `lst[0]` - access individual elements with square brackets and int index
- `for var in lst:` - **loop over a list**. On each iteration of the loop, Python points the variable *var* to the next element
- `value in lst` - boolean test if *value* is in `lst` (just like for string)

Dictionary

- Dictionary is a data structure that stores information in key-value pairs.
- While keys must be unique and immutable (like strings or numbers), values can be of any data type, whether mutable or immutable.
- This makes dictionaries ideal for accessing data by a specific name rather than a numeric position like in list.



Dictionary

```
dict = {'ca': 'California', 'ok': 'Oklahoma', 'nj': 'New  
Jersey', 'tx': 'Texas'}
```

Dictionary

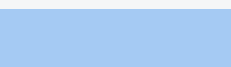
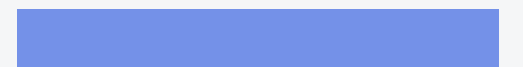
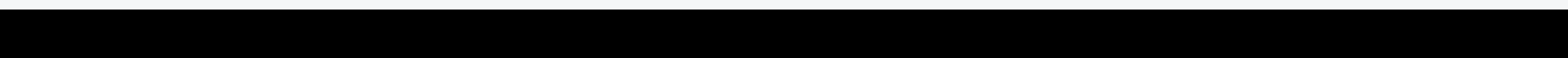
```
d = {"name": "Kat", "age": 21}
print(d["name"])

# Access using key
print(d.get("age"))
print(d.get("age", "age not given"))
```

Note: Accessing a missing key with [] raises a KeyError, while get() is safer because it returns None (or a default value) instead of an error.

Dictionary

```
d = {"a": 1, "b": 2}  
del d["a"]  
print(d)
```



Tuple

- A **Python tuple** is an ordered, immutable collection of elements.
- Think of it exactly like a list, but with one major rule: **once you create a tuple, you cannot change, add, or remove its elements.**

```
# Creating a mixed-type tuple
user_profile = ("Alice", 28, "Engineer", True)
# Accessing elements using indexes (starts at 0)
print(user_profile[0]) # Output: Alice
print(user_profile[1]) # Output: 28
```

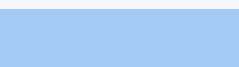
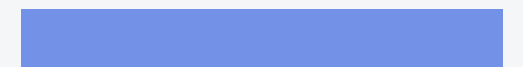
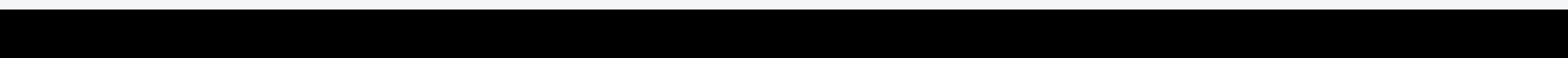
Tuple

- A very clean Python feature is "unpacking," which lets you extract tuple elements directly into separate variables.

```
# Unpacking a tuple  
dimensions = (1920, 1080)  
width, height = dimensions  
print(width)      # Output: 1920  
print(height)     # Output: 1080
```

Set

- Built-in data type
- To store an **unordered collection of unique items**.
- Sets are highly optimized for **membership testing** (checking if an item exists) and performing **mathematical operations** like unions and intersections



Key Characteristics of Sets

- **Unique Elements:** Duplicate values are automatically removed.
- **Unordered & Unindexed:** Elements do not have a defined order, and you cannot access them using indexes (like `my_set[0]`).
- **Mutable Structure:** You can add or remove items after the set is created.
- **Immutable Items:** Individual items inside the set must be immutable and hashable (e.g., strings, numbers, or tuples). You cannot put lists or dictionaries inside a set.

1. Creating Sets

You can create a set using curly braces `{ }` or the `set()` constructor.

Creating a set with elements

```
fruits = {"apple", "banana", "cherry", "apple"}
```

```
print(fruits)
```

Output: {'banana', 'apple', 'cherry'} (Duplicates removed!)

Creating an EMPTY set

Note: `{}` creates an empty dictionary. You MUST use `set()` for an empty set.

```
empty_set = set()
```

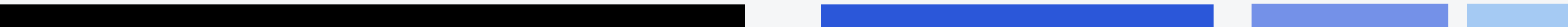
Converting a list to a set to remove duplicates numbers = [1, 2, 2, 3, 4, 4]

```
unique_numbers = set(numbers) # Output: {1, 2, 3, 4}
```

2. Adding and Removing Elements

Because sets are mutable, you can modify their contents using built-in methods.

- **.add()**: Inserts a single element.
- **.update()**: Inserts multiple elements (from a list, tuple, or another set).
- **.remove()**: Removes an element. Raises a `KeyError` if the element doesn't exist.
- **.discard()**: Removes an element but does **not** raise an error if it is missing.



2. Adding and Removing Elements

Because sets are mutable, you can modify their contents using built-in methods.

```
my_set = {1, 2, 3}

my_set.add(4)          # my_set is now {1, 2, 3, 4}

my_set.update([5, 6, 7]) # my_set is now {1, 2, 3, 4, 5, 6, 7}

my_set.remove(3)        # my_set is now {1, 2, 4, 5, 6, 7}

my_set.discard(99)      # Does nothing, no error raised
```

3. Mathematical Set Operations

Operation	Method	Operator	Description
Union	<code>set1.union(set2)</code>	<code>set1 set2</code>	All unique elements from both sets.
Intersection	<code>set1.intersection(set2)</code>	<code>set1 & set2</code>	Only elements present in both sets.
Difference	<code>set1.difference(set2)</code>	<code>set1 - set2</code>	Elements in <code>set1</code> but not in <code>set2</code> .
Symmetric Difference	<code>set1.symmetric_difference(set2)</code>	<code>set1 ^ set2</code>	Elements in either set, but not both.

3. Mathematical Set Operations

```
a = {1, 2, 3}
```

```
b = {3, 4, 5}
```

```
print(a | b) # Union: {1, 2, 3, 4, 5}
```

```
print(a & b) # Intersection: {3}
```

```
print(a - b) # Difference: {1, 2}
```

```
print(a ^ b) # Symmetric Difference: {1, 2, 4, 5}
```

4. Membership Testing

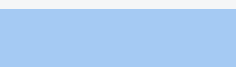
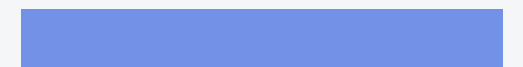
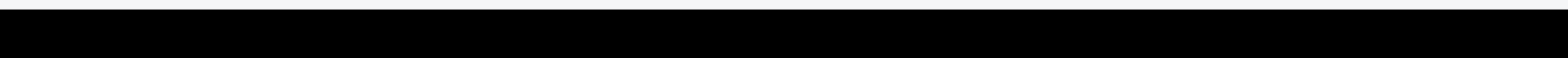
```
guests = {"Alice", "Bob", "Charlie"}  
  
if "Alice" in guests:  
    print("Alice is invited!")
```

5. Frozensets (Immutable Sets)

If you need a set that cannot be changed after creation, use `frozenset()`.

```
immutable_set = frozenset([1, 2, 3])  
immutable_set.add(4)  
# This will throw an AttributeError
```

Python Syntax



Code Block in C++

```
#include <iostream>

int main() {

    for (int i = 0; i < 5; i++) {

        std::cout << "Number: " << i << "\n";

    }

    return 0;

}
```

Code Block in Python

```
for i in range(5):  
    print(i)
```

```
for i in range(5):  
    print(i)  
    if (i == 2):  
        print("2")
```

Use of Colon and Indentation

Indentation: 4 spaces or Tab

Formatted Output

- The most common and modern way to mix text and variables is using **f-strings** (formatted string literals).
- Just put an f before the opening quote and place variables or expressions inside {} curly braces.

```
age = 25
```

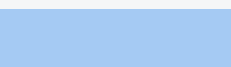
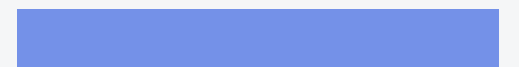
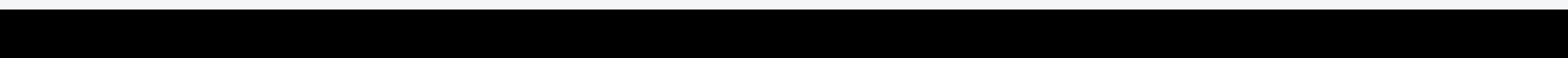
```
# Standard f-string
```

```
print(f"I am {age} years old.")
```

```
# Expressions inside f-strings
```

```
print(f"Next year, I will be {age + 1}.")
```

Python Loop



For Loop

for

item

in

iterable

:

- **item**: A placeholder variable that takes the value of the current element during each iteration.
- **Iterable/sequence**: The collection you want to loop through.

Loop Through a List or String

for**item****in****iterable****:**

Looping through a list

```
fruits = ["apple", "banana", "cherry"]
```

```
for fruit in fruits:
```

```
    print(fruit)
```

Looping through a string

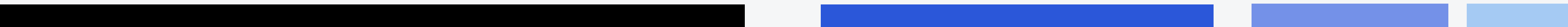
```
for letter in "Python":
```

```
    print(letter)
```

range()

[range\(\) function](#) is used with for loops to generate a sequence of numbers. It can take one, two or three arguments:

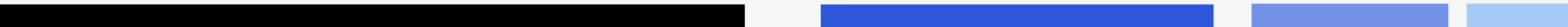
- **range(stop)** generates numbers from 0 to stop-1.
- **range(start, stop)** generates numbers from start to stop-1.
- **range(start, stop, step)** generates numbers from start to stop-1, incrementing by step.



range()

[range\(\) function](#) is used with for loops to generate a sequence of numbers. It can take one, two or three arguments:

- **range(stop)** generates numbers from 0 to stop-1.
- **range(start, stop)** generates numbers from start to stop-1.
- **range(start, stop, step)** generates numbers from start to stop-1, incrementing by step.



Loop a specific number of items

for

item

in

iterable

:

```
for i in range(5):  
    print(i)
```

```
for i in range(2, 10, 2):  
    print(i)
```

Get Both Index and Value using enumerate()

for**index, item****in****enumerate(****iterable****):**

```
colors = ["red", "green", "blue"]
```

```
for index, color in enumerate(colors):
```

```
    print(f"Index {index} is {color}")
```

Combine Two Lists using zip()

for**item1, item2****in****zip(****Iterable1, iterable2****):**

```
names = ["Alice", "Bob", "Charlie"]
```

```
scores = [85, 92, 78]
```

```
for name, score in zip(names, scores):
```

```
    print(f"{name} scored {score}")
```

Combine Two Lists using zip()

for**item1, item2****in****zip(****iterable1, iterable2****):**

- There is no limit to how many iterables you can pass.

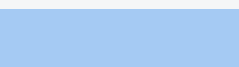
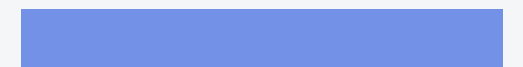
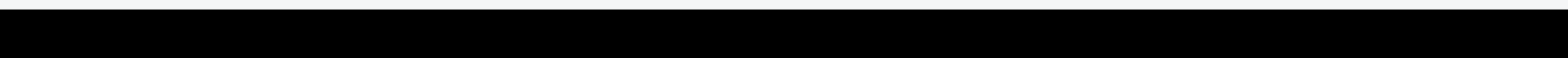
```
for name, score, grade in zip(names, scores, grades):
```

Zippping with enumerate():

for**item1, item2****in****enumerate(zip(****iterable1, iterable2****):**

```
for i, (name, score) in enumerate(zip(names, scores)):
    print(f"Rank {i+1}: {name} has {score}")
```

Python Conditional



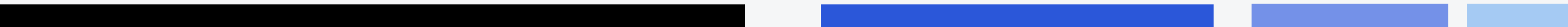
Conditional

```
traffic_light = "yellow"
if traffic_light == "red":
    print("Stop!")
elif traffic_light == "yellow":
    print("Slow down!")
elif traffic_light == "green":
    print("Go!")
else:
    print("Invalid light color.")
```

Inline conditional

```
status = "Adult" if age >= 18 else "Minor"
```

Python Function



Function

Parts of a function definition:

- **def** - A function begins with the word def
- **name** - def is followed by the function's name
- **parenthesis** - the name is followed by a pair of parenthesis and a colon ():
- **body lines** - Indented within the def are the "body" lines of code which make up the function.

```
def foo():  
    x = 1  
    y = 2  
    z = 3  
    i = 0
```

```
def foo(x, y):  
    x = x - y  
    return x + 1
```

Function

```
def add_numbers(a, b):  
    return a + b  
result = add_numbers(5, 7)  
print(result)
```

Output: 12

Default Arguments:

```
def power_up(base, exponent=2):  
    return base ** exponent  
  
print(power_up(4))  
# Uses default exponent: 4^2 = 16  
  
print(power_up(4, 3))  
# Overrides default: 4^3 = 64
```

Function

Keyword Arguments:

You can pass arguments using the parameter names, meaning the order doesn't matter.

```
def describe_pet(animal, name):  
    print(f"I have a {animal} named {name}.")  
  
describe_pet(name="Buddy", animal="dog")
```

**Thank
You!**

